Park-AI: Scalable, Affordable

&

Real-Time Urban Parking Management

Prepared By Hasan, Abhishek Sharma, Mubassir Sapa

April 7th, 2025

📄 Table of Contents

1. Problem Statement

Finding parking in large areas like malls, schools, or office buildings is often time-consuming and frustrating. Drivers waste time and fuel circling around just to find an empty spot, which can lead to delays or missed appointments.

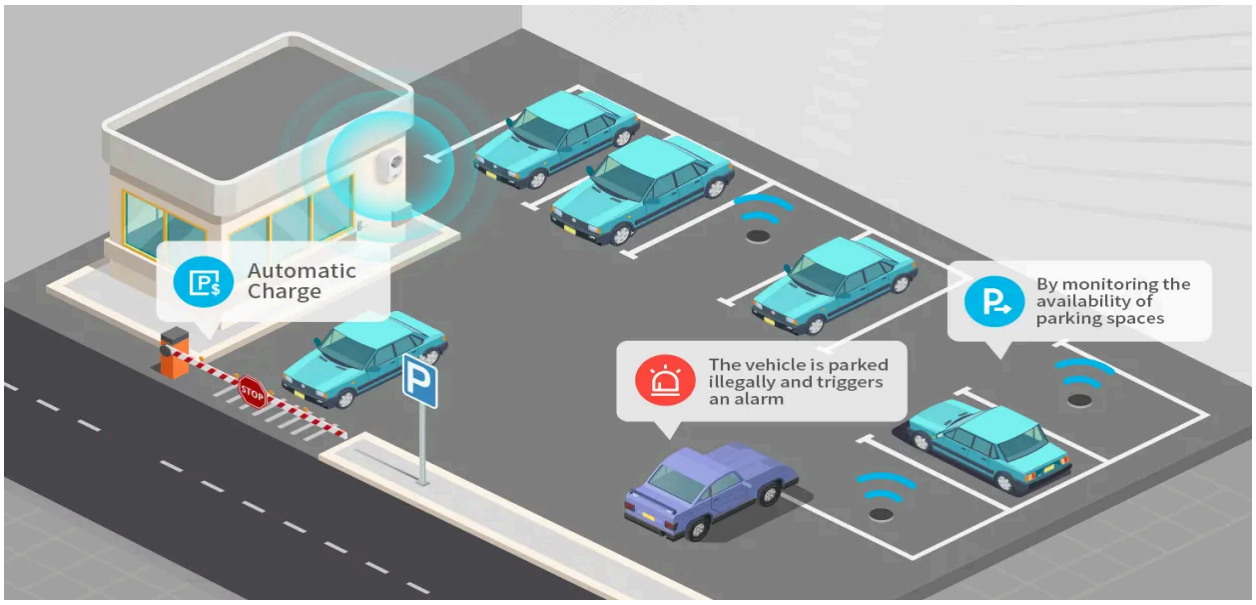
We observed this problem firsthand during exam season. Many students prefer arriving close to exam time to stay focused or revise last-minute notes. But due to limited parking and no way to check availability in advance, they often spend critical minutes searching for a spot — adding stress or causing delays. Some even miss their exam window.

This problem is made worse by the lack of a system to check parking availability remotely. Most places still rely on drivers physically entering the lot and searching, which adds unnecessary congestion and frustration during peak hours.

2. Background Research

Several smart parking systems have been developed over the years, such as sensor-based setups, cloud-based AI cameras, and ANPR (Automatic Number Plate Recognition), each with its pros and cons.

Sensor-Based Systems:
These use ground or overhead sensors to detect occupancy. The data is sent to a central system to show availability. While accurate, they're costly and require one sensor per spot, along with digging, wiring, and regular maintenance—making them hard to deploy in older infrastructure.



ANPR Systems:
ANPR reads license plates to manage access, track durations, and handle payments. It's great for gate control but doesn't help detect empty spots. It also struggles in poor visibility conditions.

While these methods are smart, they can be expensive and difficult to apply in existing buildings or campuses. Many setups would need better connectivity, camera upgrades, or rewiring. This highlights the need for a more affordable solution that uses existing infrastructure like basic CCTV—reducing both cost and complexity.

3. Proposed Solution

Our solution uses existing CCTV footage, processed through a locally trained AI model (like YOLOv8 or YOLOv11), to detect parking spot availability. Instead of relying on cloud services or installing new sensors, we run the model on a nearby computer, Raspberry Pi, or even a microcontroller, keeping costs and power usage low. In many cases, existing hardware is enough.

A key benefit is that no major infrastructure changes are needed—we reuse existing power and CCTV setups. Additional features like LED indicators or LCDs are optional and low-cost.

This system is easily adaptable for malls, schools, or offices. For example, a mall could display parking info via QR code, while schools might use a small web or mobile app. Owners could also install screens or LEDs to show available spots. Over time, the system can collect parking data for future planning and analysis.

In our demo, we used an ESP32-CAM as the camera, a laptop running a model trained on 3,000+ labeled parking images, and an FRDM-K66F board to control a servo motor (gate), LEDs, and an LCD. More technical details are explained in the next section.

4. Advantages Over Existing Solutions

Our approach offers several advantages compared to traditional sensor-based or cloud-based parking systems:

- **Low Cost**: We avoid using expensive sensors or cloud AI services. By running the AI model locally and using existing CCTV cameras, the setup cost is much lower.

- **No Major Infrastructure Changes**: The system works with what's already available — cameras, computers, and power. There's no need to dig, rewire, or install new hardware unless someone wants extra features like lights or screens.

- **Offline Capability**: Since the AI model runs locally, there's no need for constant internet access or cloud connectivity, making it more reliable and secure.

- **Flexible Output Options**: Users can choose how they want to display the result — LCD screen, website, mobile app, or LED indicators. This makes it useful for different environments like malls, schools, and offices.

- **Energy Efficient**: Using microcontrollers instead of full systems can save on electricity, especially in long-term use.

- **Easy to Scale and Maintain**: It's easier to update the code or model without changing physical hardware. Also, replacing a camera or board is cheaper than fixing wired sensors.

4.Development Process & Challenges

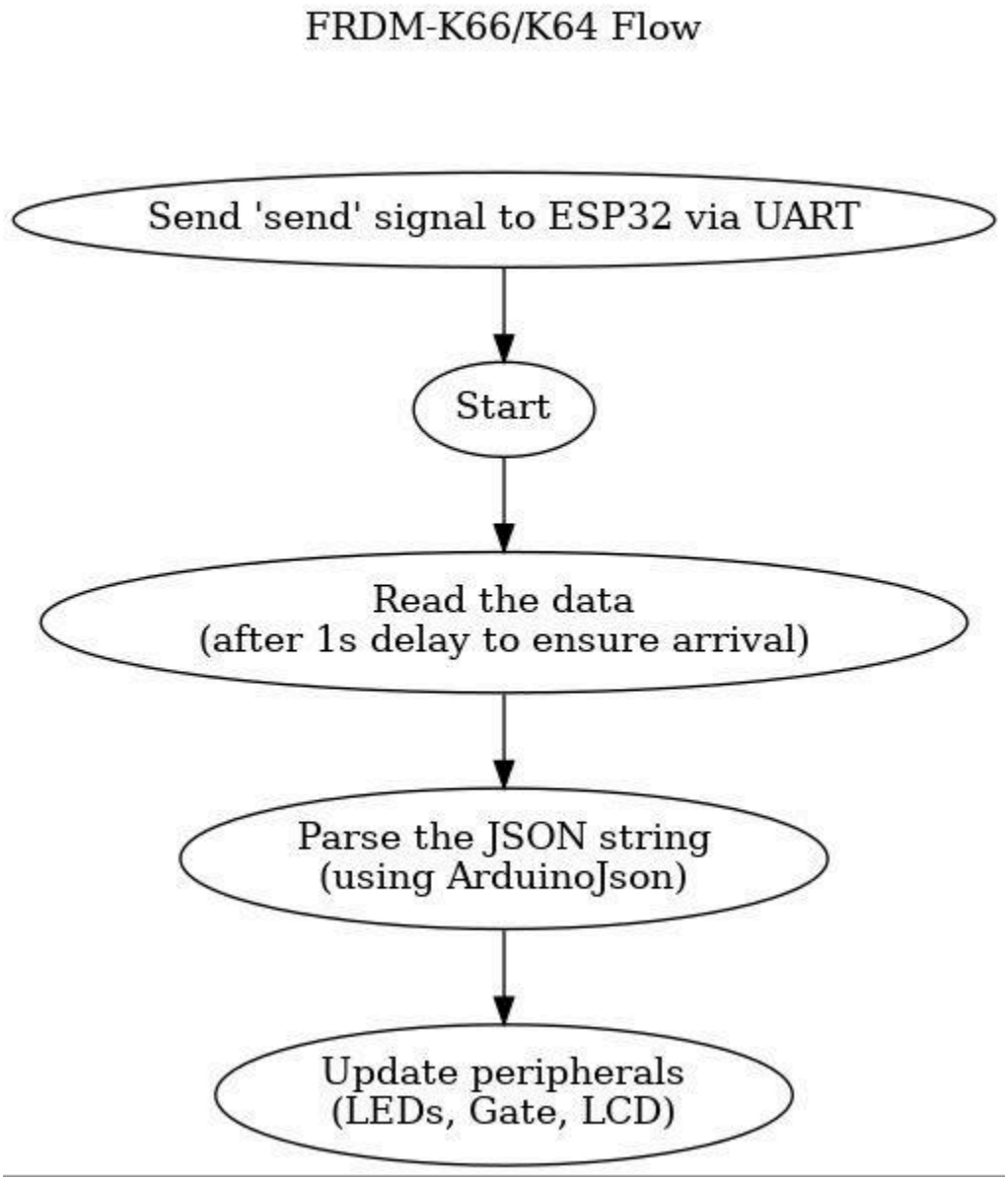
How We Came Up with the Solution

As mentioned in the problem statement, blindly looking for parking wastes time and fuel. Having a quick way to know if there's parking available would save both. For example, in Seneca, an app or website showing real-time parking availability could help students and teachers plan better. So, we thought—most parking areas already have CCTV. Why not use that footage, run it through a model or LLM to detect parking status, and use that data to inform drivers? That's how the core idea of our project was formed.

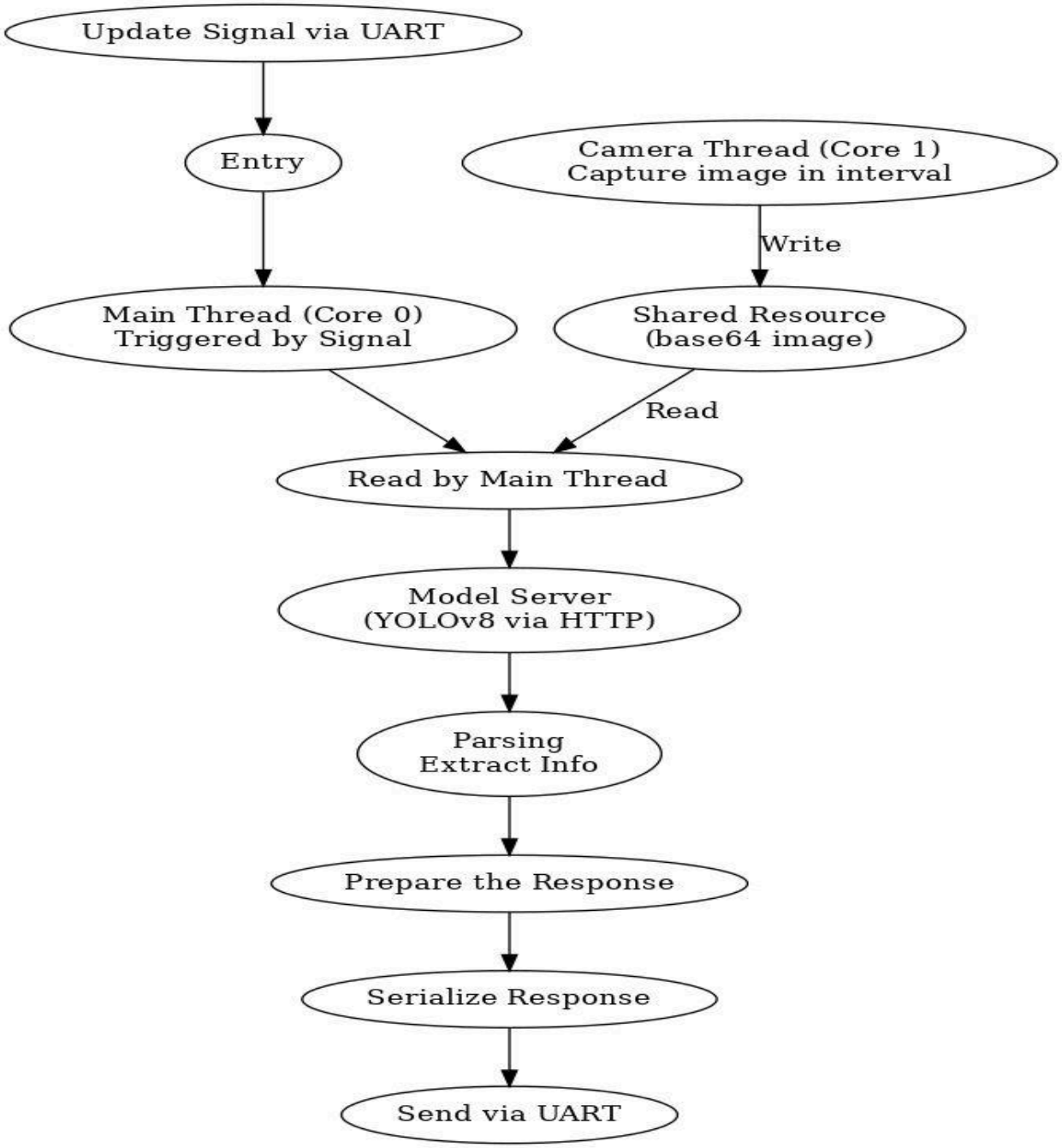
Our Final Solution

Since we didn't have access to CCTV footage, we had two options: use a pre-recorded video or a live camera. We decided to go with the ESP32-CAM module. Initially, we tried using GPT-based LLMs, but the results were inconsistent and unreliable (we explain this in challenges). Finally, we trained our own object detection model using YOLOv8 on 2K+ labeled images. The whole solution was divided into three main modules:

1. **FRDM-K66F**: It controls all the peripherals like LCD, LEDs, and the servo motor. It sends signals to the ESP32-CAM to fetch and process image data, and then updates the peripherals based on the result.



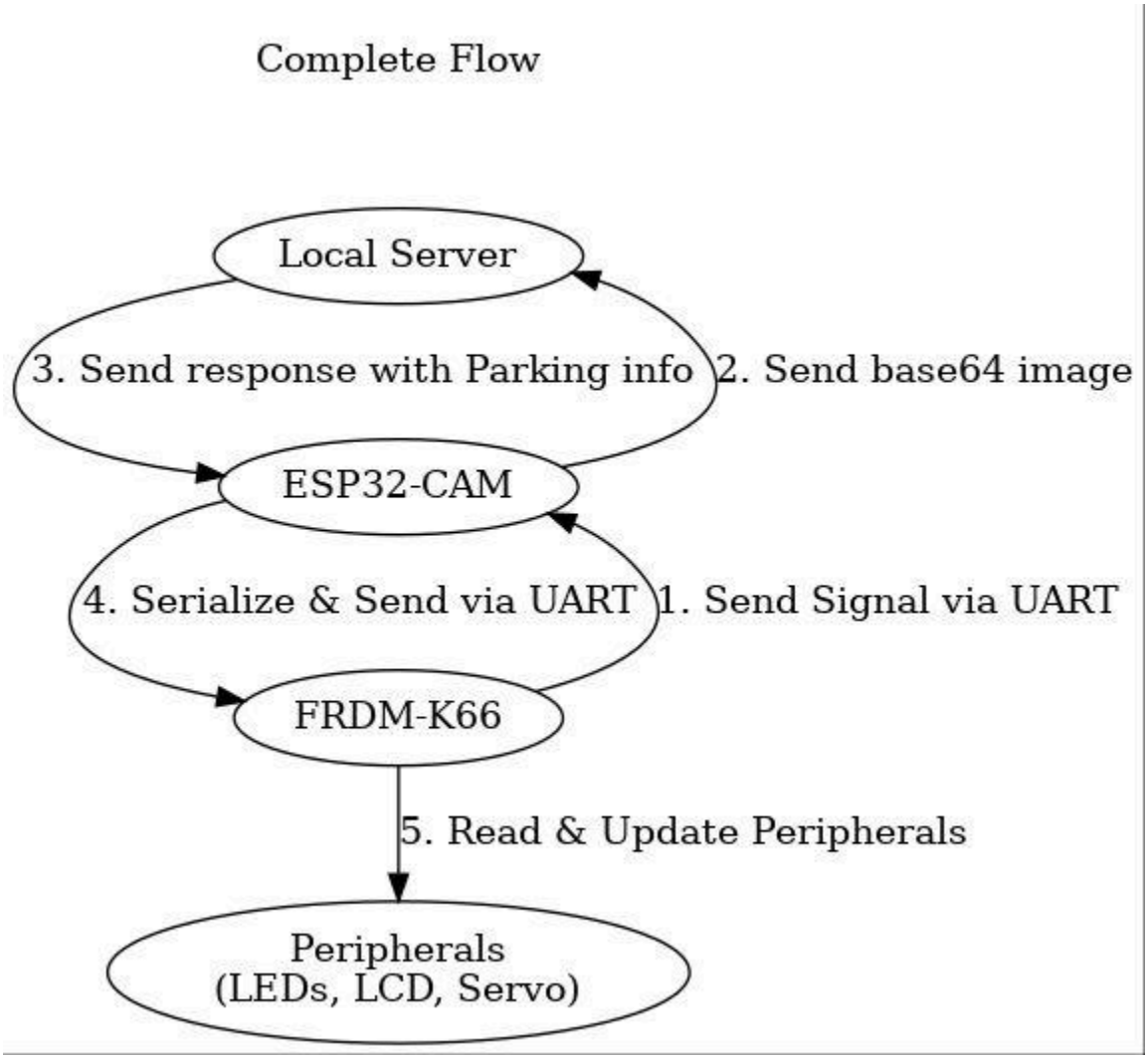
2. **ESP32-CAM**: This module uses the OV2640 camera and built-in Wi-Fi to capture images and send them to a local server (running Flask and YOLOv8). To avoid blocking the main thread, a separate thread runs on core1 to capture an image every second and stores it in a shared resource. Semaphore is used to handle reading/writing between the threads. Once the image is processed by the server, the result is sent to the FRDM via UART.



3. **Model Server**: YOLOv8 was first trained in Google Collab with labeled images split into training, testing, and validation sets. The server is built using Flask, which receives base64 images, processes them using the `.pt` YOLOv8 model, annotates the results with OpenCV, and returns JSON data about which parking spots are empty or occupied.

Flow

The flow starts with FRDM sending a signal via UART → ESP32 receives the signal and fetches the latest image from the shared resource (handled by the background thread) → The image is sent to the local server → The server processes it with the YOLOv8 model and returns the results (including annotated image and parking status) → ESP32 sends this data back to the FRDM → FRDM updates the peripherals accordingly.



To ensure consistency among server and the 2 microcontrollers, we used a data structure like

`{ status: true/false, data: { ... } }`.

If `status` is `true`, `data` contains parking info in JSON; if `false`, `data` contains an error message.

Challenges Faced

One major challenge was using GPT-based LLMs at the beginning. Due to their non-deterministic nature and limited context windows, the outputs were inconsistent. We even tried sequential multi-agent systems, but they were slow and sometimes got stuck in loops. So, we switched to a more reliable and fast solution: a fine-tuned pre-trained YOLOv8 model.

Another challenge was UART synchronization. After sending a signal, ESP32 doesn't get the data immediately since the image must be processed first. This delay caused sync issues, so we added a small delay in FRDM after sending data to allow ESP32 time to respond with new results.

Lastly, ESP32's main thread was getting blocked during image capture, causing delays and incorrect data. We solved this by using parallelism—running image capture on core1 while keeping the main logic on core0, improving responsiveness and accuracy.

5.Health, Safety, Environmental, and Societal Considerations

Our project is designed to be safe and non-invasive. It uses camera-based detection without any physical interaction, ensuring no health or safety risks to users. Environmentally, it helps reduce unnecessary driving time and fuel consumption by providing real-time parking availability, which can lower CO_2 emissions and traffic congestion.

We also aimed to use existing infrastructure, like CCTV cameras, which reduces overall cost and avoids the need for digging or installing new sensors. This approach makes the solution easier to deploy, more sustainable, and suitable for integration into urban or institutional environments, benefiting both individuals and the community.

6.Future Work

In the future, we plan to integrate our system with real-world parking lots by using existing CCTV infrastructure. This will make the solution more practical and scalable. We also aim to improve our model's accuracy and robustness by training it on more diverse datasets. Additionally, we plan to develop a user-friendly web or mobile app that will display real-time parking availability, making it easily accessible for students, staff, and the general public.

7.References

[1] Cogniteq, “Smart Parking Systems: What They Are and Why They’re Beneficial,” *Cogniteq Blog*, 2023. [Online]. Available: https://www.cogniteq.com/blog/smart-parking-systems-what-they-are-and-why-theyre-beneficial

[2] Intuz, “IoT in Smart Parking Management: Benefits and Challenges,” *Intuz Blog*, 2022. [Online]. Available: https://www.intuz.com/blog/iot-in-smart-parking-management-benefits-challenges

[3] Nortech Control, “What Is ANPR and How Does It Work?,” *Nortech Blog*, 2021. [Online]. Available: https://blog.nortechcontrol.com/what-is-anpr

[4] **Last Minute Engineers**, “ESP32-CAM Pinout Reference: Which GPIO pins should you use?”, *LastMinuteEngineers.com*. [Online]. Available: https://lastminuteengineers.com/esp32-cam-pinout-reference/

[5] **Last Minute Engineers**, “Servo Motor With Arduino Tutorial - How Servo Motor Works, How To Control It, Arduino Code”, *LastMinuteEngineers.com*. [Online]. Available: https://lastminuteengineers.com/servo-motor-arduino-tutorial/

[6] **DroneBot Workshop**, “ESP32-CAM Video Streaming and Face Recognition”, *YouTube*, Apr. 2020. [Online]. Available: https://www.youtube.com/watch?v=visj0KE5VtY&t=1622s&ab_channel=DroneBotWorkshop

[7] **Technobyte**, “How to interface a seven segment display with Arduino Uno”, *Technobyte.org*. [Online]. Available: https://technobyte.org/seven-segment-arduino-uno-interfacing/

[8] **Arm Mbed**, “FRDM-K66F Development Board”, *os.mbed.com*. [Online]. Available: https://os.mbed.com/platforms/FRDM-K66F/

[9] **Ultralytics**, “Train YOLOv8 on a Custom Dataset”, *YouTube*, Jan. 2023. [Online]. Available: https://www.youtube.com/watch?v=LNwODJXcvt4&t=1s&ab_channel=Ultralytics

8.Work Links

1. **Github Repo:** https://github.com/MohHasan1/ParkAI
2. **Quick Demo:** https://drive.google.com/file/d/1MSu5mgmAo936-EsWqRB523nmbo4QnHnl/view?usp=sharing
3. **Canva** Link:https://www.canva.com/design/DAGeqgreN_8/P4sL1DMihljzPCytqGvYfg/edit?utm_content=DAGeqgreN_8&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

13. Progression Table:

Date Range	Task/Activity	Status	Allocated
Feb 25 - Mar 5	ESP32 CAM configuration & image capture programming	Done	(Mubassir)
Mar 5 - Mar 10	Gemini Model 2.0 API integration & AI response parsing	Done	(Hasan)
Mar 10 - Mar 15	UART communication setup (ESP32 <-> K64) & LED indicator system	Done	(Abhishek)
Mar 16 - Mar 21	Fine-tune YOLOv8 model using Roboflow Universe (parking-specific data)	Done	(Hasan)
Mar 16 - Mar 21	Optimize ESP32 CAM image quality (mounting, better lighting)	Done	(Abhishek)
Mar 22 - Mar 26	Integrate LCD display and LEDs (show available spots count)	Done	(Mubassir)
Mar 27 - Mar 31	Implement servo motor (simulate parking barrier)	Done	(Hasan)
Apr 1 - Apr 4	System-wide optimization (latency reduction, multi-device sync)	Done	(Abhishek)
Apr 5 - Apr 7	Final testing, debugging, and milestone report preparation	Done	(Abhishek, Hasan, Mubassir)